ChurnGuard AI — AutoML, XAI & DevOps

Three-page report · IBM Telco Customer Churn · FastAPI + React + SHAP · all numbers below are from **docs/training_run.log** (EXITCODE=0)

Problem & dataset

Predict whether a telecom customer will churn (binary classification; churn is the positive class). The IBM Telco sample has 7,043 accounts and 20 columns. After dropping `customerID` and 11 rows with a blank `TotalCharges`, the cleaned data has **7,032 rows and 19 features** (3 numeric, 16 categorical) with a churn rate of **0.266**. The split is stratified 80/20: **5,625 train / 1,407 hold-out** rows (`random_state=42`). F1 is the primary metric because the classes are imbalanced and a missed churner costs more than a wasted retention offer; precision, recall and ROC-AUC are reported alongside it.

Step 2 · Optuna Bayesian optimisation (baseline)

train.py · Optuna baseline (20 TPE trials)

```
2. Baseline model (LogisticRegression + Optuna Bayesian optimisation)
   untuned reference: CV F1 = 0.5960 (+/- 0.0224)  holdout F1 = 0.6099
     trial  0: CV F1 = 0.5889  C=0.07459  class_weight=None  fit_intercept=True
     trial  1: CV F1 = 0.6275  C=0.006025  class_weight=balanced  fit_intercept=False
     trial  2: CV F1 = 0.3632  C=0.001267  class_weight=None  fit_intercept=True
     ... 16 more trials ...
     trial 19: CV F1 = 0.6313  C=0.03015  class_weight=balanced  fit_intercept=False
   Optuna: best CV F1 = 0.6338 after 20 trial(s) in 10.5s
   Optuna best params: {'C': 1.1462107403425035, 'class_weight': 'balanced', 'fit_intercept': False, 'max_iter': 2000, 'random_state': 42}
   tuned baseline: CV F1 = 0.6338 (+/- 0.0029)  holdout F1 = 0.6061
   holdout F1 = 0.6061  precision = 0.4901  recall = 0.7941  roc_auc = 0.8351
```

Optuna's TPE sampler tuned regularisation `C`, `class_weight` and `fit_intercept` with 3-fold stratified CV on the training partition only. Best CV F1 **0.6338**; hold-out F1 **0.6061** (precision 0.4901, recall 0.7941, ROC-AUC 0.8351). The untuned reference scored 0.5960 CV / 0.6099 hold-out.

Step 3 · PyCaret vs FLAML under the same 60-second budget

train.py · PyCaret vs FLAML on the same holdout

```
3. AutoML model comparison (60s budget per engine)
   PyCaret candidates: lr, dt, rf, et, gbc (holdout scored on 1407 rows, budget 60s)
   PyCaret CV F1: LR 0.5970 | GBC 0.5859 | RF 0.5615 | ET 0.5260 | DT 0.4966
   PyCaret winner: LogisticRegression (holdout F1 = 0.6082, 60.2s)
   FLAML winner: rf (holdout F1 = 0.5844, 60.0s, budget 60s)
   Tool comparison (same stratified holdout)
tool                 budget_s  holdout_f1  notes
Optuna (baseline LR)  60        0.6061      Optuna (20 trials, 10.5s)
PyCaret               60        0.6082      LogisticRegression
FLAML                 60        0.5844      rf
```

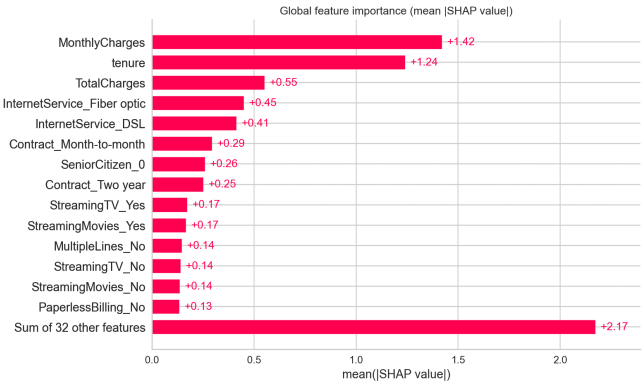
PyCaret's internal split is disabled (`train_size=0.999`) and `compare_models` is capped at 60 seconds; FLAML is capped at `time_budget=60` seconds. Both are scored on the exact same 1,407-row hold-out, so the comparison is apples-to-apples. PyCaret selected logistic regression (0.6082); FLAML spent its budget on a small random forest (0.5844).

Step 4 · Critical evaluation: deliberate leakage

train.py · deliberate leakage probe

```
4. Deliberate leakage probe
[WARNING] Model with leaky feature achieved F1: 1.0000
[ACTION] Leaky feature dropped; final model retrained without 'future_churn_indicator'.
   holdout F1 with / without the probe: 1.0000 / 0.6099
```

A synthetic `future_churn_indicator` column was appended to copies of the frames as an exact copy of the label. The probe model reached **F1 = 1.0000**, which is the expected signature of leakage. The script prints a warning, drops the column, and retrains; the column never enters the real feature frame or the deployed pipeline. The same audit checks identifier removal, duplicate rows, fit/transform isolation, `TotalCharges ~= tenure x MonthlyCharges` redundancy (r = 1.000, known at prediction time, so redundancy rather than target leakage), class balance, and fairness slices.



*Global SHAP summary (mean |SHAP|, 200 hold-out rows). Top drivers: **MonthlyCharges 0.1790**, **tenure 0.1561**, **InternetService 0.1242**.*

GET /model-card

Dataset, metrics (baseline vs deployed), intended use, known limits, features, explainability, fairness slices, leakage checks/probe, AutoML runs and validation. Keys: `model, dataset, metrics, intended_use, known_limits, features, explainability, fairness, leakage_checks, leakage_probe, automl_tools, validation`.

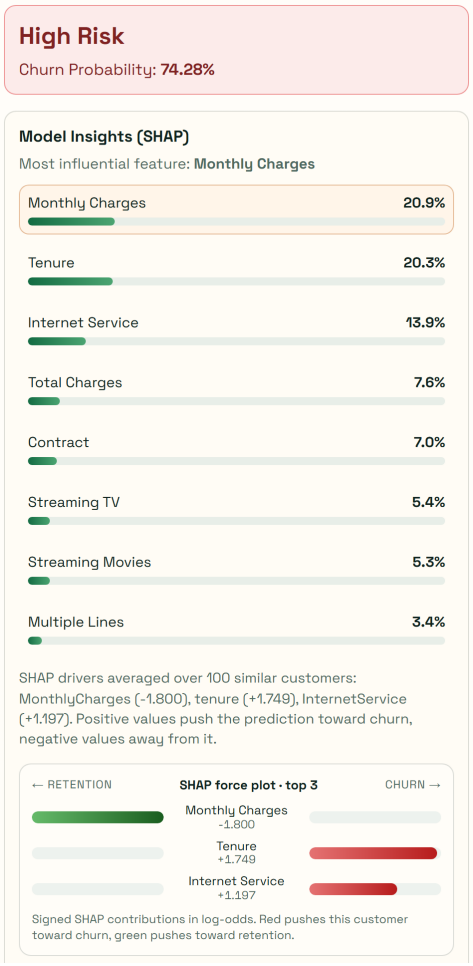
Honest result

Tuned AutoML hold-out F1 **0.6054** vs **0.6061** for the Optuna baseline (**-0.0007**), well inside the CV standard deviation (0.0216). AutoML adds a selection process, tuning record and model card, not free accuracy; no improvement is claimed.

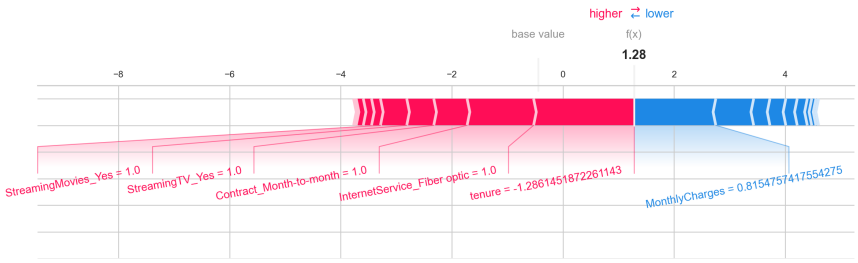
Executable validation: 6/6 PASS — Optuna completed · FLAML vs PyCaret comparison completed · leakage detected and dropped · SHAP plots generated · /metrics and /model-card present · K8s and Terraform files exist.

Explainable AI — from global drivers to this customer

Every POST `/predict` returns the probability, the original normalised importance, signed `top_factors` and an English explanation. The React UI renders the signed values as a SHAP force plot.



Live React UI (headless Chrome, real API): High Risk, 74.28%, importance bars and the signed force plot. The form is above; the optional-profile section and model meta are below the crop.



SHAP force plot for the highest-risk hold-out customer (`shap.plots.force(..., matplotlib=True)`). Red pushes the prediction toward churn; blue pushes it toward retention.

How to read the GUI

The live React screenshot (left) shows **High Risk · 74.28%**, the normalised importance bars, and the signed force plot built from `top_factors`. Each row is a feature; the bar grows from the centreline toward retention (green, left) or churn (red, right), and the printed number is the log-odds contribution. This answers "why this customer?" rather than only "which features matter globally?".

Innovation · kNN marginalisation

A retention agent rarely has all 19 Telco fields. Instead of filling missing inputs with one made-up "typical" row or per-column medians, the API averages the model over the **100 training customers closest to the supplied inputs in tenure and monthly charges** (distance in standard deviations). Supplied values are applied to every one of those rows. This estimates `P(churn | supplied)` by marginalising over the missing dimensions, so the score and its SHAP values stay on-manifold. The response names the inferred fields in `assumed_features` and reports `aggregation = {method: neighbour_average, rows: 100}`; a single-row fast path is used when every input is supplied. SHAP values are averaged over the same look-alike rows, so the force plot explains the number actually returned.

The same explanation from the API

Request: tenure 2, MonthlyCharges 95, month-to-month, 4 support calls, fibre optic, electronic check, paperless billing, no tech support. Response: **High Risk · 74.28%**. *SHAP drivers averaged over 100 similar customers: MonthlyCharges (-1.800), tenure (+1.749), InternetService (+1.197). Positive values push the prediction toward churn, negative values away from it.*

Feature	SHAP (log-odds)	Direction
MonthlyCharges	-1.7995	toward retention
tenure	+1.7493	toward churn
InternetService	+1.1970	toward churn

The kNN path inferred 11 fields and reported `aggregation.rows = 100`. SHAP values describe model behaviour, not causal effects: they say which inputs the model relied on, not what would happen if a customer changed a feature.

DevOps — CI/CD, containers, orchestration & observability

The same repository ships a blocking lint/test gate, a multi-stage image, Kubernetes manifests with two replicas and Terraform for Azure. Monitoring is built in, not bolted on.

CI gate · lint and tests before the image is built

`.github/workflows/deploy.yml` runs a blocking `ci` job on every push: install `requirements-dev.txt`, run **flake8** (style errors fail the build), then **pytest** (35 tests; the suite is hermetic). The `deploy` job has `needs: ci`, so it cannot start unless both pass.

ci job · flake8 + pytest (local mirror)

```
$ flake8 . --count --statistics
0

$ pytest -q
35 passed, 12 warnings in 2.47s

[PASS] ci job: flake8 + pytest gate the Docker build
```

Local mirror of the CI job (the public Actions screenshot is attached separately when the run completes).

Multi-stage Dockerfile

```
FROM python:3.10-slim AS builder
RUN apt-get update && apt-get install \
    -y --no-install-recommends build-essential
RUN python -m venv /opt/venv \
 && /opt/venv/bin/pip install -r requirements.txt

FROM python:3.10-slim AS runtime
COPY --from=builder /opt/venv /opt/venv
COPY --chown=app:app . .
RUN useradd --uid 1001 --gid 1001 app
USER app
HEALTHCHECK CMD python -c "..."
CMD ["uvicorn", "main:app", "--port", "8000"]
```

Kubernetes (2 replicas)

```
# k8s/deployment.yaml
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 1001
      containers:
        - name: api
          readinessProbe:  /model-info
          livenessProbe:   /
```

Terraform (Azure RG + ACR)

```
# terraform/main.tf
resource "azurerm_resource_group" "main" {
  name     = "${var.project}-rg"
  location = var.location
}

resource "azurerm_container_registry" "main" {
  name                = var.acr_name
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  sku                 = "Basic"
  admin_enabled       = true
}
```

Observability · Prometheus at /metrics

`prometheus-fastapi-instrumentator` is wired in `main.py`. `GET /metrics` exposes request counts, status codes, latency histograms and in-progress requests; the API also keeps structured request logs and a drift endpoint. The excerpt below is a real capture from the running service.

GET /metrics · Prometheus exposition

```
$ curl -s http://127.0.0.1:8000/metrics | grep -E 'http_requests|duration'
# HELP http_requests_total Total number of requests by method, status and handler.
# TYPE http_requests_total counter
http_requests_total{handler="/",method="GET",status="200"} 1.0
http_requests_total{handler="/model-card",method="GET",status="200"} 1.0
http_requests_total{handler="/predict",method="POST",status="200"} 2.0
http_requests_total{handler="none",method="GET",status="404"} 7.0
http_requests_total{handler="none",method="OPTIONS",status="200"} 1.0
http_request_duration_seconds_count{handler="/",method="GET"} 1.0
http_request_duration_seconds_count{handler="/model-card",method="GET"} 1.0
http_request_duration_seconds_count{handler="/predict",method="POST"} 2.0
http_request_duration_seconds_count{handler="none",method="GET"} 7.0
http_request_duration_seconds_count{handler="none",method="OPTIONS"} 1.0
```